AI 301 · UNIT 4 · TEST & SUBMIT

Submit & Land It

The whole tool is yours.

AI 301 · WELCOME

Welcome to Unit 4: Submit & Land It

1. **Turn on** your camera
2. **Answer** tonight's question in the chat

TONIGHT'S QUESTION · IN THE CHAT

In your past job/internship/project experiences, what made a code change landable?

OUR COMMUNITY

As a community, we will...

* **Be present**, physically and mentally
* **Actively build** an inclusive community and space
* **Practice** curiosity and adopt a learning mindset
* **Ask** for help when we need it
* **Keep** our cameras on

Your Feedback



Glows

+ Practical course that teaches real SWE skills
+ Time consuming but rewarding



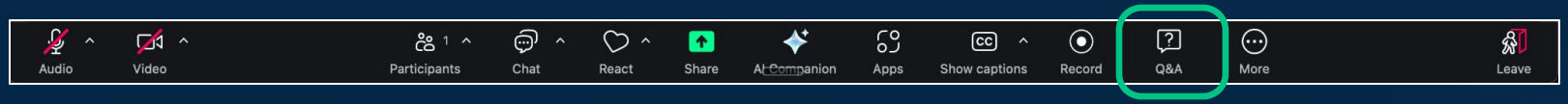
Grows

- Clearer instructions
- More time on sharing industry experience
- More breakout collaboration

ASK ANYTIME

Add your questions to the Q&A

- Our Tech Fellow team adds answers throughout the lesson
- Chat questions may be missed or redirected to the Q&A

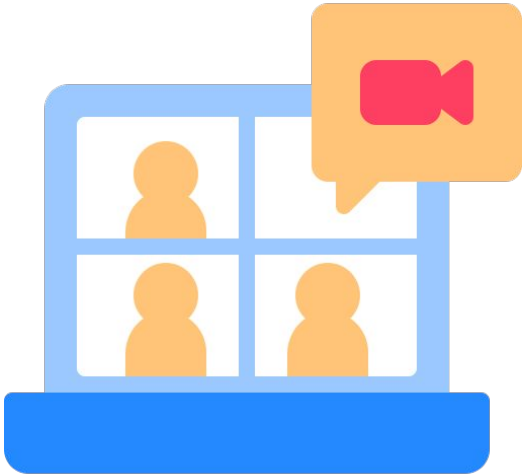


Q&A, on the right

Announcements & Reminders

Don't Forget Our Standards of Engagement!

- **Cameras on:** visible in frame (head + shoulders in frame, well-lit)
- Calling in from a **computer**
- Calling in from a **singular, stable location** with stable access to internet



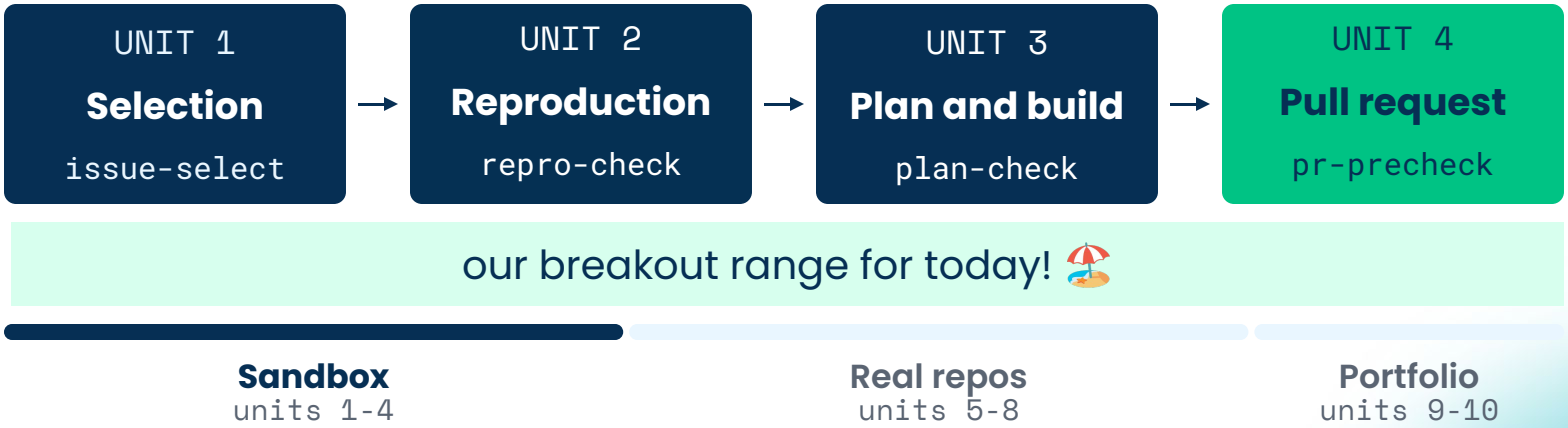
AI 301 · UNIT 4 · 120 MINUTES

Agenda

01	**Intro**	continuity: from branch to pull request	10 min
02	**Lecture**	the PR as argument, pr-precheck, and forks	45 min
03	**Break**		5 min
04	**Activity**	Read It Like a Maintainer, in breakout rooms	50 min
05	**Wrap-up**	what's due, and Beat 2 opens	10 min

Where this session sits

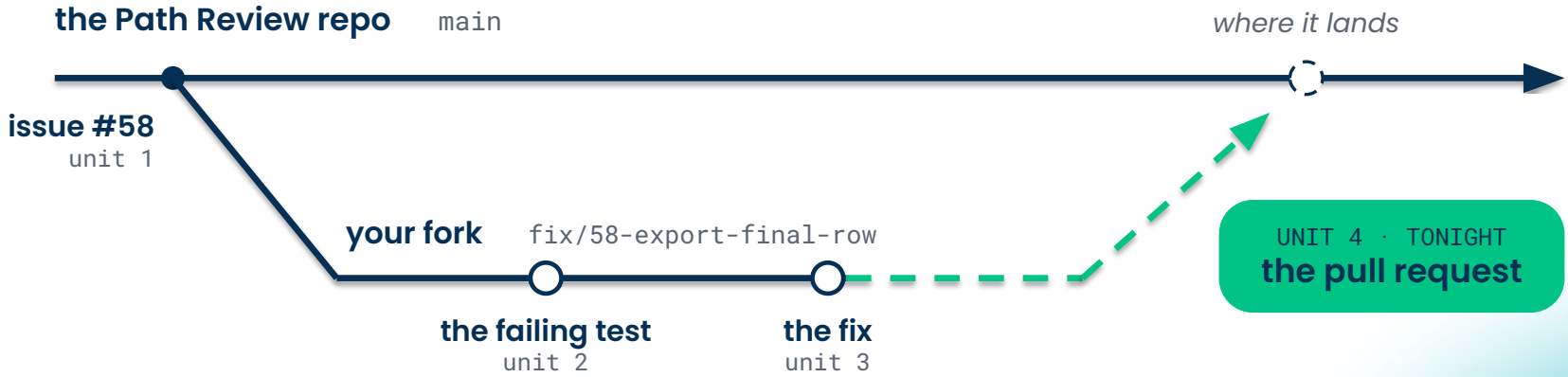
we're in 🏖️ the sandbox! 🏖️ and after… we explore the world outside.



FROM BRANCH TO PULL REQUEST

Your first pull request

A **pull request** asks the shared repo to take your branch, with everything a maintainer needs to judge it without you in the room.



Tonight you build pr-precheck, clear the eval bar a fourth time, and **open a PR in the sandbox.**

CODEPATH*ORG

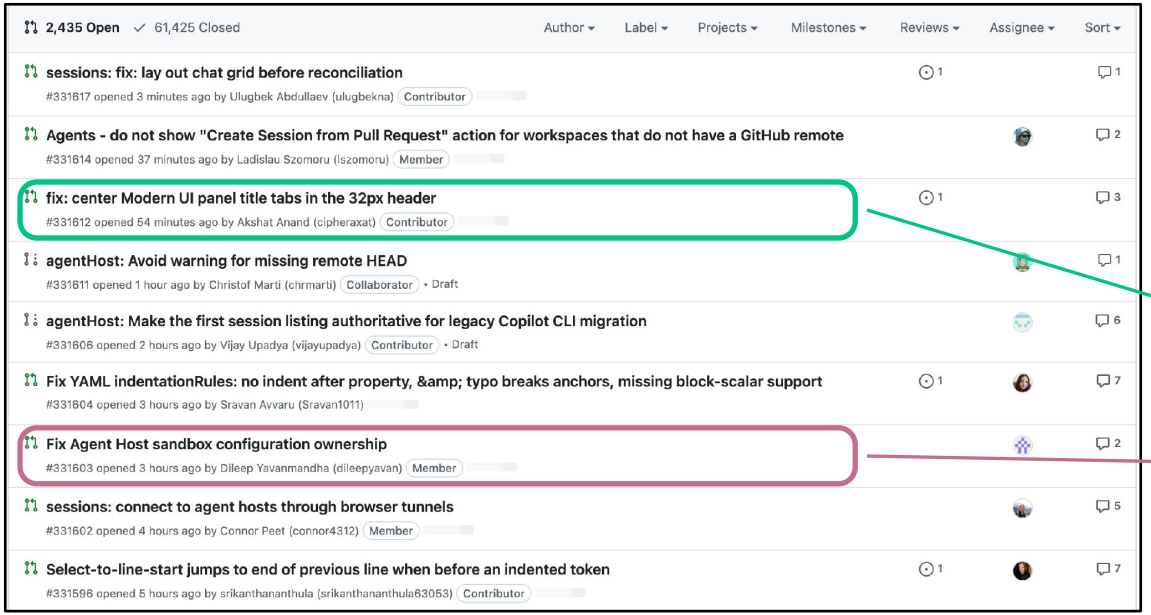
LECTURE · PART 1 OF 5

The first read

How a PR spends a maintainer's thirty seconds.

THE MAINTAINER'S READ

Thirty seconds



github.com/microsoft/vscode · 2,435 open pull requests

30 seconds

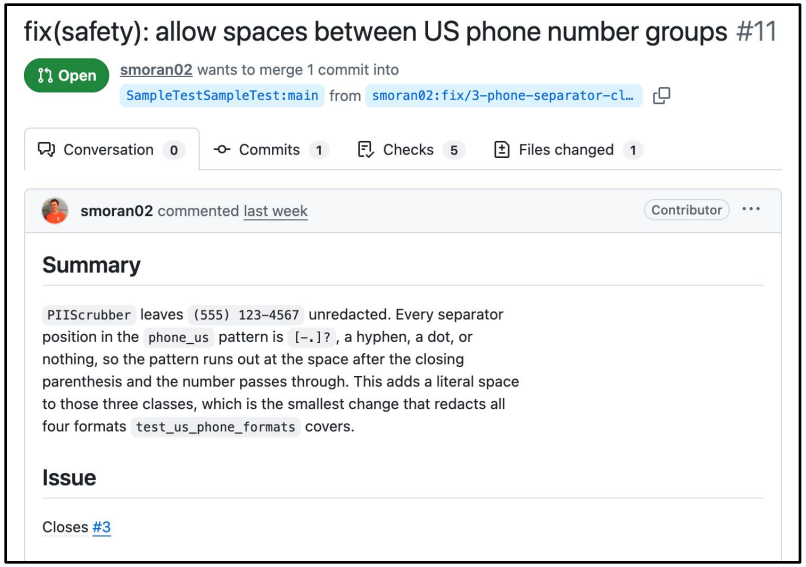
what gets read before the maintainer decides

says what and where
worth the next minute

which config? what broke?
the thirty seconds end here

ANATOMY

The PR is a proposed case



the title	**what changed, and where?**
the description	**why should I believe it?**
the commits	**how did it get here?**
the diff	**what exactly moved?**
the evidence	**how do I see it work?**

PATH REVIEW'S PR TEMPLATE

The template, filled in

Six sections, and every one gets real content. Here is each one for #58.

Section	What goes in it, for #58
Summary	Export skipped the last row. The loop now writes every row.
Issue	Closes #58
Changes	One bullet per change: export.py loops over every row, not rows[:-1].
Testing	Before and after output (41 rows, then 42), plus each check's result.
Screenshots / Demo	N/A, and why: it's a command-line change.
Notes for Reviewers	How you used AI, and anything that differs from your plan.

CODEPATH*ORG

LECTURE · PART 2 OF 5

Self-review

You are the first stranger to read your own diff.

READING A DIFF

How to read a diff

A diff shows only what a PR changes, file by file.

```
--- a/export.py
+++ b/export.py
@@ -12,3 +12,3 @@ def export_rows(rows, out):
     writer = csv.writer(out)
-    for row in rows[:-1]:
+    for row in rows:
         writer.writerow(row)
```

which file: a/ is before, b/ is after

where: this part starts at line 12

no sign: unchanged, shown for context

minus: a line removed

plus: a line added

Your own branch's diff: `git diff main...HEAD` (add `--stat` for just the list of files)

SELF-REVIEW

Read it like a stranger

your branch · the diff, read cold

```
@@ export.py
-   for row in rows[:-1]:
+   for row in rows:
        write(row)

@@ names.py
+   def tidy_header(s):
+       return s.strip().title()
```

the plan never mentions this change

1 · matches the plan?

2 · would I understand this cold?

3 · would I ask for changes here?

names.py adds a helper the plan never asked for, so question 1's answer is no, **and no is the finding**

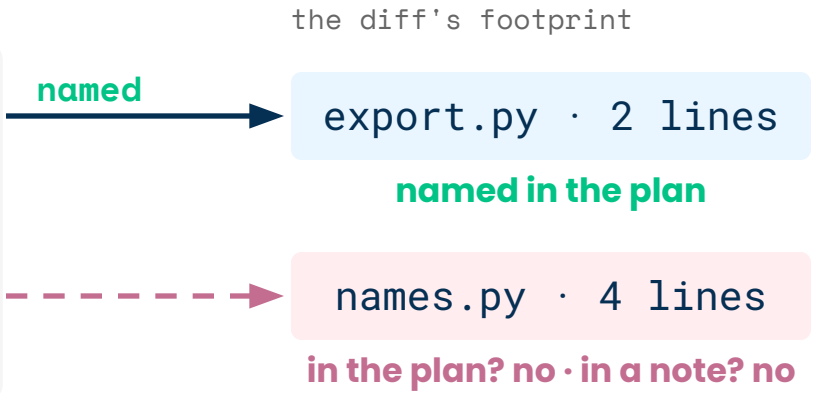
The cheapest review is the one you do first.

WHAT FAILS A PR · 1 OF 4

Silent drift

the posted plan · #58

will	include the final row
files	export.py
won't	touch helpers or format



NO NOTE	silent drift: the PR fails, however good the code is
NOTE RECORDED	in plan.md, restated in the description: it can pass

WHAT FAILS A PR · 2 OF 4

Not tested

BEFORE

```
$ python demo.py export
rows in:      42
rows written: 41
```

AFTER

expected-after, stated

```
$ python demo.py export
rows in:      42
rows written: 42
```

plus the repo's own checks where the plan or repo asks, outcome visible

`pytest · 312 passed` `lint · clean` `build · ok`

Evidence names the observable change. "Tests pass" names nothing.

WHAT FAILS A PR · 3 OF 4

Unreviewable



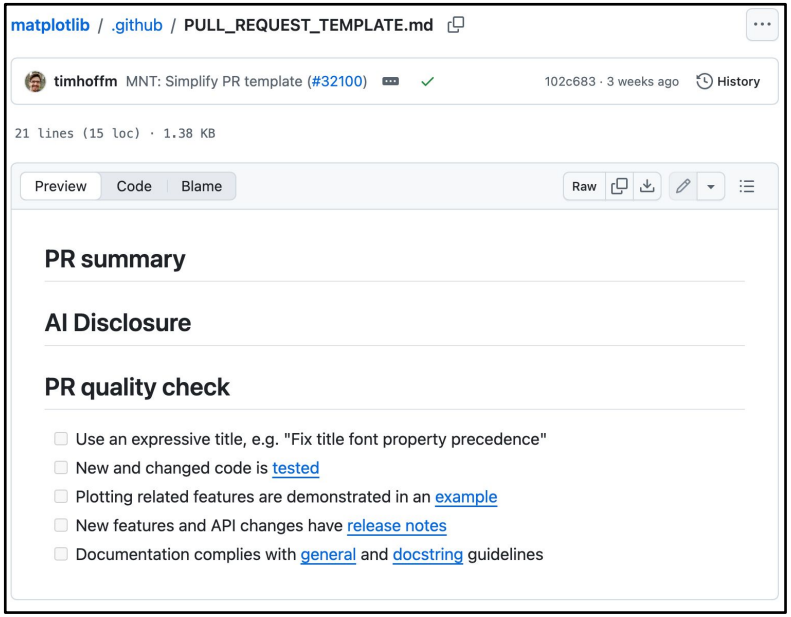
3 lines of fix, 40 of noise

- the fix · 3 lines
- debris · the other 40 lines

The fix is invisible. A reviewer's thirty seconds go to the debris and run out first.

Sweep the debris and the same PR is 3 lines that read in one breath.

The standards wall



matplotlib/matplotlib · PR template

Use of Generative AI

Generative AI tools are evolving rapidly and can be helpful. As with any tool, the resulting contribution is the responsibility of the human contributor. We expect dedicated and authentic engagement in our community. In particular when using AI, carefully consider what and how to communicate, question results, think things through thoroughly and make well-informed decisions.

matplotlib.org · contributing guide

Every stated ask is a check in your rubric.

Does course work disclose? Always.

this workflow is AI-assisted by design, so the disclosure section always gets real content, in your own words

CODEPATH*ORG

LECTURE · PART 3 OF 5

Your tool

Every file of pr-precheck is yours now.

THE ONE FILE YOU ONLY READ

Why CONTRACT.md is fixed

Read it before you write SKILL.md. The eval harness and unit 7's ship-gate both run your tool, and they rely on these rules.

CONTRACT.md fixes	Why it can't change
The file layout	Both programs find your files at these exact paths.
Two modes: live and eval	One tool checks your own PR and grades the practice PRs.
Verdict: accept or reject	Every verdict can be compared with the gold labels.
A JSON block, last in the reply	Both programs read the verdict there. No block, no verdict.
Refuse if a file is empty	No guessed verdicts from an unfinished tool.

THE PARTS BIN

You already built the parts

tools/pr-precheck/ · seven files

- SKILL.md NEW
- rubric.md
- procedure.md
- voice-guide.md
- scope.md
- references/
  - evidence-guide.md
- CONTRACT.md

the frame is yours

pattern from units 1 + 3

stages from unit 3

paste forward, extend it

fill the Repo line

families from units 2 + 3

fixed · read it first

This week the whole folder is yours.

Units 1-3 handed you staff frames to fill in. Here every file starts empty.

You write four from your parts, extend your voice guide, and fill in scope.md's Repo line. CONTRACT.md you only read.

ONE GRADED PACKAGE

The tool runs

pr-precheck · pkg-demo · eval mode

matches-plan	P	the diff is the plan's export fix
no-silent-drift	F	**extra hunk: names.py; no note anywhere**
⋮		the rest of the card: yours to write

HOLD — deciding check: **diff exceeds plan; no deviation note**

```json
{"item": "pkg-demo",
 "checks": [ ... ],
 "verdict": "reject"}
```

last in the reply, always

every grade cites the package's own lines

CODEPATH*ORG

LECTURE · PART 4 OF 5

The eval

Same bar, fourth time.

THE EVAL

Same bar, fourth time: 18/20

- **18/20 to pass.**
- **Every category lands at least once:** clear accept, silent drift, not tested, unreviewable, standards wall.
- **Unlimited retries.** about $0.25 a package, $4-5 for a full run.

Your eval write-up carries the other points. Four things, in order:

- your eval runs, first to last
- one disagreement with the gold labels, named by package id
- the check you changed because of it, quoted
- what that change cost elsewhere in the run

CODEPATH*ORG

LECTURE · PART 5 OF 5

Fork to merge

Where your PR comes from, and where it goes.

FORKS

Three copies of one repo

You can't push to the Path Review repo. So you push to your own copy, and a pull request asks the Path Review repo to take your branch.

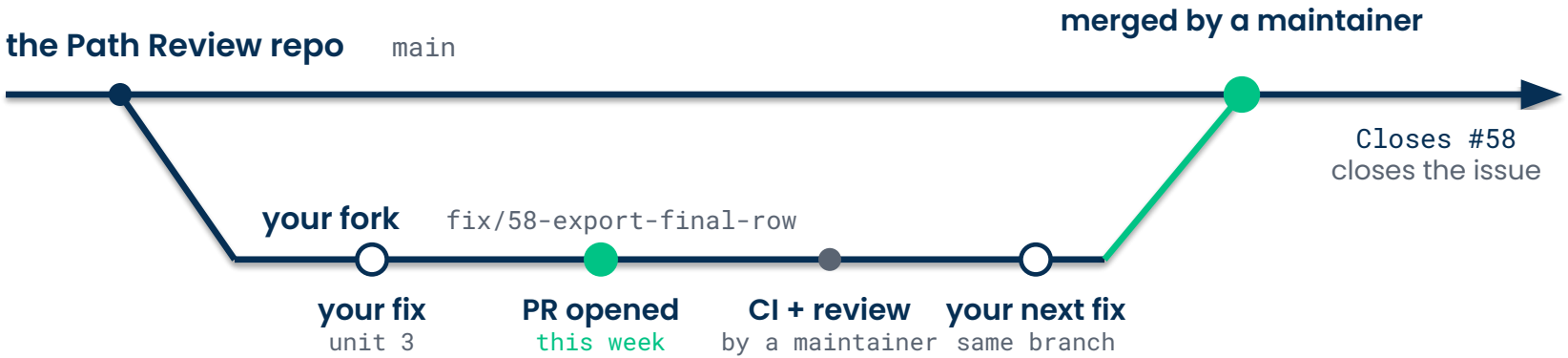


Unit 2 did steps 1 and 2 once. This week is 3 and 4: push your branch to origin, then open the PR into the Path Review repo's main.

THE MERGE FLOW

After you open it

Your PR stays open while it's checked and reviewed. Each push to your branch adds to the same PR, so you never open a second one.



Only a maintainer can merge. In the sandbox, your PR is graded as you opened it, so a review, a merge, or a close never changes your score.

THE SANDBOX PR

Open it right

01	**Push your branch to your fork**	`git push -u origin fix/58-export-final-row`
02	**Open the PR from your fork**	the Compare & pull request button, on your fork
03	**Base it on the Path Review repo**	`base: main`
04	**Fill the whole template**	every section real · `Closes #58`
05	**Record the link**	`beat-1-sandbox/unit-4/pull-request.md`

Opening the PR is the graded outcome: six of the eighteen points, scored before the diff is read.

AFTER THE BREAK · TONIGHT'S ACTIVITY

Read It Like a Maintainer

Your group checks one practice PR, then fixes it. First you're the maintainer. Then you switch sides and become the contributor.

Every file the PR changes Phase 1 · you're the maintainer	→	**`a plan line`** that asked for it
Every claim the PR makes Phase 1 · you're the maintainer	→	**`proof`** a diff line or real output
Every no you found Phase 2 · you're the contributor	→	**`a fix`** in the code or description

Your group's tables become the start of your homework rubric.

CODEPATH*ORG

BREAK · 5 MINUTES

Take five

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



THE ACTIVITY · 50 MINUTES · BREAKOUT ROOMS

Read It Like a Maintainer

Setup · 5 min · pick a captain first

1. Copy the worksheet, share it as Editor — captain
2. Paste the link in the Zoom chat, share your screen — captain
3. Work in the captain's copy, not your own — everyone

Getting started · nothing from lecture needed

Phase 1 hands you calib-03, a practice PR, to check as the maintainer. Every step is on the Activity tab.

BEFORE ROOMS OPEN | open the Activity tab, where the worksheet link is

Breakout

ONE GOOD THING

Let's celebrate your wins

Reflect on today's session. Maybe you got a question answered, overcame a coding challenge, or learned how to do something completely new.

IN THE CHAT | share one good thing you learned or experienced today

SESSION SURVEY · 3 MINUTES

Tell us how tonight went

Your feedback makes the next session better,
and we read all of it.

Take the survey · link in the chat



BEFORE YOU GO

One thing, due Sunday

Your course repo link the same one you've submitted every unit

- `tools/pr-precheck/`: every file of your tool, `SKILL.md` included
- `beat-1-sandbox/unit-4/pull-request.md`: your PR link, branch name, and the four eval fields
- `beat-1-sandbox/unit-4/eval-run.txt`: one complete run, written by `--save-run`
- `beat-1-sandbox/unit-3/plan.md`: upload it again if you added Deviations

50 PTS full rubric on the Grading Rubrics page, nothing hidden

unlimited retries before Sunday · submit the link on the Assignment tab

CODEPATH*ORG

UNIT 5 · NEXT UNIT

Into the wild

Beat 2 starts next session: all of GitHub. Your unit 1 skill grows into a scout.

TONIGHT
build the tool · clear the bar · open the PR · submit

BEFORE YOU GO

Final reminders

- Build the tool · clear the bar · open the PR · submit
- Coworking Cafe at 7pm PT again this week!
- **Next Unit:** Into the wild

